Privacy Act Compliance Report

OWASP Juice Shop

Report ID: c1ea165e-611f-43d6-83fb-88d4b9ea4c85
Date: 24 April 2026
Framework: APP / NDB (Privacy Act 1988)
Scanner: AttestLens v0.2.0
Target URL: https://demo.owasp-juice.shop/

CONFIDENTIAL

Executive Summary

Overall Grade: A (92/100)
Risk Level: MEDIUM

Moderate risk identified. 13 APP controls need attestation. Address findings to improve compliance posture.

Consultant Narrative

This assessment shows weak breach readiness (0/100). 1 APP/NDB control failed, 13 remain unresolved, and 2 critical/high technical findings should be prioritised before external delivery.

Evidence basis: This run used scanner evidence only. Organisational process controls still require operator attestation.

Remediation focus: Start with "Fix: Missing Content-Security-Policy Header" because it addresses APP_11 and improves the highest-priority breach-readiness factors.

Breach Readiness

Score: 0/100
Level: WEAK
Confidence: 37%
Evidence basis: scanner_only

Breach readiness is weak at 0/100 with 37% confidence under a scanner-only review. Primary factor: APP_11 Security failed.

Top readiness factors:
CRITICAL / questionnaire: APP_11 Security failed (APP_11)
HIGH / scanner: Missing Content-Security-Policy Header (APP_11)
HIGH / scanner: Missing Strict-Transport-Security Header (APP_11)
MEDIUM / assessment_gap: Questionnaire attestation not completed
MEDIUM / scanner: Missing Referrer-Policy Header (APP_11)

Finding Summary

Critical: 0
High: 2
Medium: 5
Low / Info: 4

APP Control Status

APP_1 — Open & Transparent: UNKNOWN (100/100, 0 findings)
APP_2 — Anonymity: UNKNOWN (100/100, 0 findings)
APP_3 — Collection: UNKNOWN (100/100, 0 findings)
APP_4 — Unsolicited Info: UNKNOWN (100/100, 0 findings)
APP_5 — Notification: UNKNOWN (100/100, 0 findings)
APP_6 — Use & Disclosure: UNKNOWN (100/100, 0 findings)
APP_7 — Direct Marketing: UNKNOWN (100/100, 0 findings)
APP_8 — Cross-Border: UNKNOWN (100/100, 0 findings)
APP_9 — Gov Identifiers: UNKNOWN (100/100, 0 findings)
APP_10 — Data Quality: UNKNOWN (100/100, 0 findings)
APP_11 — Security: FAIL (49/100, 11 findings)
APP_12 — Access: UNKNOWN (100/100, 0 findings)

APP_13 — Correction: UNKNOWN (100/100, 0 findings)
NDB — Data Breach Notification: UNKNOWN (100/100, 0 findings)

Technical Findings

[HIGH] Missing Content-Security-Policy Header

fe25fe7f836c67ab • AL-URL-001 • Security Headers

Content-Security-Policy prevents XSS, clickjacking, and other code injection attacks by specifying approved sources of content.

APP references: APP_11

Suggested fix: Add a Content-Security-Policy header to your server configuration:

Fix prompt: Add a Content-Security-Policy header to your server configuration:

For Next.js (next.config.js):
```js
const cspHeader = `
  default-src 'self';
  script-src 'self' 'unsafe-eval' 'unsafe-inline';
  style-src 'self' 'unsafe-inline';
  img-src 'self' blob: data:;
  font-src 'self';
  object-src 'none';
  base-uri 'self';
  form-action 'self';
  frame-ancestors 'none';
`

module.exports = {
  async headers() {
    return [{ source: '/(.*)', headers: [{ key: 'Content-Security-Policy', value: cspHeader }] }]
  }
}
```

This addresses APP 11.1 — taking reasonable steps to protect personal information from misuse, interference, and loss.

[HIGH] Missing Strict-Transport-Security Header

accda2c93269c694 • AL-URL-002 • Security Headers

HSTS forces browsers to always use HTTPS, preventing protocol downgrade attacks and cookie hijacking.

APP references: APP_11

Suggested fix: Add a Strict-Transport-Security header to your server configuration:

Fix prompt: Add a Strict-Transport-Security header to your server configuration:

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```

For Express:
```js
app.use(require('helmet')());
// or specifically:
app.use((req, res, next) => {
  res.setHeader('Strict-Transport-Security', 'max-age=31536000; includeSubDomains; preload');
  next();
});
```

This addresses APP 11.1 — protecting personal information during transmission.

[MEDIUM] Missing Referrer-Policy Header

85c39219d2705fc5 • AL-URL-005 • Security Headers

Referrer-Policy controls how much referrer information is shared with other sites. Without it, browsers may send full URLs including sensitive path/query parameters.

APP references: APP_11

Suggested fix: Add a Referrer-Policy header:

Fix prompt: Add a Referrer-Policy header:

```
Referrer-Policy: strict-origin-when-cross-origin
```

For Next.js:
```js
module.exports = {
  async headers() {
    return [{ source: '/(.*)', headers: [{ key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' }] }]
  }
}
```

This addresses APP 6.1 — controlling disclosure of personal information via referrer headers.

[LOW] Missing Permissions-Policy Header

cfd111829c8cc401 • AL-URL-006 • Security Headers

Permissions-Policy controls which browser features and APIs the page can use (camera, microphone, geolocation, etc.).

APP references: APP_11

Suggested fix: Add a Permissions-Policy header:

Fix prompt: Add a Permissions-Policy header:

```
Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=()
```

For Express:
```js
app.use((req, res, next) => {
  res.setHeader('Permissions-Policy', 'camera=(), microphone=(), geolocation=(), payment=()');
  next();
});
```

This addresses APP 11.1 — restricting access to sensitive browser APIs.

[INFO] Missing X-XSS-Protection Header (Legacy)

6c42e9faf7357fb3 • AL-URL-007 • Security Headers

X-XSS-Protection is a legacy header for older browsers. Modern browsers rely on Content-Security-Policy instead. Only needed if supporting IE or old browsers.

APP references: APP_11

Suggested fix: X-XSS-Protection is a legacy header. Modern browsers use Content-Security-Policy instead.

Fix prompt: X-XSS-Protection is a legacy header. Modern browsers use Content-Security-Policy instead.

If you need to support older browsers:
```
X-XSS-Protection: 1; mode=block
```

However, the preferred approach is to use Content-Security-Policy, which provides stronger XSS protection. Consider this a low-priority fix unless you have specific IE/legacy browser requirements.

This addresses APP 11.1 — XSS prevention.

[MEDIUM] Missing Cross-Origin-Opener-Policy Header

f71ffb2cb9eec67c • AL-URL-008 • Security Headers

COOP isolates your document from cross-origin documents opened via window.open or links, preventing side-channel attacks like Spectre.

APP references: APP_11

Suggested fix: Add a Cross-Origin-Opener-Policy header:

Fix prompt: Add a Cross-Origin-Opener-Policy header:

```
Cross-Origin-Opener-Policy: same-origin
```

For Express:
```js
app.use((req, res, next) => {
  res.setHeader('Cross-Origin-Opener-Policy', 'same-origin');
  next();
});
```

This addresses APP 11.1 — protecting against cross-origin information leaks.

[MEDIUM] Missing Cross-Origin-Resource-Policy Header

953649ebf553c5ac • AL-URL-009 • Security Headers

CORP prevents cross-origin resources from being loaded by other sites, mitigating speculative side-channel attacks.

APP references: APP_11

Suggested fix: Add a Cross-Origin-Resource-Policy header:

Fix prompt: Add a Cross-Origin-Resource-Policy header:

```
Cross-Origin-Resource-Policy: same-origin
```

For Express:
```js
app.use((req, res, next) => {
  res.setHeader('Cross-Origin-Resource-Policy', 'same-origin');
  next();
});
```

This addresses APP 11.1 — preventing cross-origin resource access.

[LOW] Missing Cross-Origin-Embedder-Policy Header

3117cca359e1d166 • AL-URL-010 • Security Headers

COEP requires all loaded resources to opt-in to being loaded, enabling use of SharedArrayBuffer and preventing data leaks.

APP references: APP_11

Suggested fix: Add a Cross-Origin-Embedder-Policy header:

Fix prompt: Add a Cross-Origin-Embedder-Policy header:

```
Cross-Origin-Embedder-Policy: require-corp
```

Note: This can break third-party resources that don't set CORP headers. Use `credentialless` for a softer approach:

```
Cross-Origin-Embedder-Policy: credentialless
```

This addresses APP 11.1 — preventing cross-origin data leakage.

[MEDIUM] Wildcard CORS Configuration

015423d4a29fb795 • AL-URL-013 • CORS

Access-Control-Allow-Origin is set to "*", allowing any origin to make cross-origin requests. This can expose user data to malicious websites.

APP references: APP_11

Suggested fix: Restrict CORS to specific trusted origins:

Fix prompt: Restrict CORS to specific trusted origins:

```typescript
// Instead of:
res.setHeader('Access-Control-Allow-Origin', '*');

// Use:
const allowedOrigins = ['https://yoursite.com', 'https://app.yoursite.com'];
const origin = req.headers.origin;
if (allowedOrigins.includes(origin)) {
  res.setHeader('Access-Control-Allow-Origin', origin);
}
```

This addresses APP 11.1 (Security) and APP 6.1 (Use/Disclosure).

[MEDIUM] innerHTML Assignment Detected

a9b9649d5fa44caa • AL-JS-004 • JS Security

Direct innerHTML assignment was found. If the assigned content includes untrusted data, this creates an XSS vulnerability. React's JSX and other frameworks auto-escape content by default.

APP references: APP_11

Suggested fix: Replace innerHTML with safer DOM manipulation:

Fix prompt: Replace innerHTML with safer DOM manipulation:

```typescript
// Instead of:
element.innerHTML = userInput;

// Use:
element.textContent = userInput; // Auto-escapes HTML

// Or for structured content:
const fragment = document.createDocumentFragment();
// ... build DOM safely
element.appendChild(fragment);
```

In React, use JSX which auto-escapes:
```tsx
<div>{userInput}</div>  // Safe — auto-escaped
```

This addresses APP 11.1 (Security).

[INFO] Console Logging in Production Code

8d51e61990f867be • AL-JS-008 • JS Security

Console logging statements were found in client-side code. While not a direct security vulnerability, verbose logging can leak sensitive data (tokens, user info, API responses) to browser developer tools.

APP references: APP_11

Suggested fix: Remove or guard console logging in production:

Fix prompt: Remove or guard console logging in production:

```typescript
// Option 1: Guard with environment check
if (process.env.NODE_ENV !== 'production') {
  console.log(data);
}

// Option 2: Use a logging library that strips console in production
// Install: npm install loglevel
import log from 'loglevel';
log.setLevel(process.env.NODE_ENV === 'production' ? 'error' : 'debug');
```

```
// Option 3: Use build-time stripping (terser, babel-plugin-transform-remove-console)
// In next.config.js:
module.exports = {
  compiler: {
    removeConsole: process.env.NODE_ENV === 'production',
  },
}
```

This addresses APP 11.1 (Security) — preventing information leakage.

Remediation Playbook

Fix: Missing Content-Security-Policy Header

MEDIUM EFFORT • 1-2 hours • APP APP_11

Content-Security-Policy prevents XSS, clickjacking, and other code injection attacks by specifying approved sources of content.

Fix: Add a Content-Security-Policy header to your server configuration:

Fix: Missing Strict-Transport-Security Header

MEDIUM EFFORT • 1-2 hours • APP APP_11

HSTS forces browsers to always use HTTPS, preventing protocol downgrade attacks and cookie hijacking.

Fix: Add a Strict-Transport-Security header to your server configuration:

Review Security Headers practices

STRUCTURAL • 1+ days • APP APP_11

3 medium-severity findings in Security Headers. Requires systematic review.

Review CORS practices

STRUCTURAL • 1+ days • APP APP_11

1 medium-severity findings in CORS. Requires systematic review.

Review JS Security practices

STRUCTURAL • 1+ days • APP APP_11

1 medium-severity findings in JS Security. Requires systematic review.

This report is generated by AttestLens and does not constitute legal advice. Consult a qualified privacy professional for formal compliance assessment.

Generated by AttestLens v0.2.0 — 24/04/2026, 11:56:14 am
Deterministic hash: 2dc8a133a60f72f01a6a1bc53c35bb01